

Project Proposal

RL-PQC-VPN: A Reinforcement Learning Driven Post-Quantum Cryptographic VPN System

1. Introduction

Most VPNs today rely on classical cryptography, specifically Diffie-Hellman key exchange and elliptic curve algorithms. These are mathematically secure against classical computers, but a sufficiently powerful quantum computer running Shor's algorithm can break them. The NSA, NIST, and multiple governments have already begun mandating a transition to quantum-resistant cryptography.

The threat that makes this urgent right now is called Harvest Now, Decrypt Later (HNDL). Attackers are already recording encrypted VPN traffic today, storing it, and waiting for quantum hardware to mature so they can decrypt it retroactively. Data sent over a classical VPN in 2025 could be readable by 2035.

This project builds a VPN that addresses this threat by replacing the classical handshake with NIST-standardised post-quantum algorithms and embedding a reinforcement learning agent that learns the device's behaviour and makes intelligent, real-time decisions about which algorithm to use and when to rotate session keys. No existing VPN product combines these two capabilities.

2. Problem Statement

Current post-quantum VPN implementations treat quantum resistance as a binary switch, either on at a fixed security level, or off. NordVPN and ExpressVPN both launched PQC support in 2024-2025, but both apply it statically. There is no adaptation based on device conditions, traffic sensitivity, or detected threats. This creates three gaps:

- Static PQC is wasteful on low-powered devices and mobile hardware where running ML-KEM-1024 continuously drains battery and CPU without justification.
- Static PQC has no response mechanism for operational threats, key compromise through memory access, session key aging, or network anomalies that indicate a man-in-the-middle situation.
- No existing system uses machine-local reinforcement learning to learn per-device security policies over time.

This system addresses all three gaps.

3. Proposed Solution

We are building a complete desktop VPN application with four integrated components: a post-quantum encrypted tunnel, a reinforcement learning decision engine, a three-layer anomaly detection pipeline, and a native desktop application built with Tauri. The primary development and deployment platform is Linux (Ubuntu 22.04 LTS). Windows support is planned as a second phase.

The three team members work on clearly separated layers that connect through defined interfaces.

3.1 Post-Quantum VPN Tunnel

The VPN tunnel is built on WireGuard, the fastest and most audited modern VPN protocol. WireGuard intentionally avoids protocol agility, so we do not replace its Diffie-Hellman handshake directly. Instead, we use WireGuard's built-in Pre-Shared Key (PSK) feature: both peers derive a quantum-

safe shared secret using ML-KEM and inject it as the WireGuard PSK, which is mixed into WireGuard's handshake. This approach is used in production by Mullvad and Tailscale.

To remain secure against both classical and quantum adversaries, we use a hybrid key exchange combining X25519 (classical Diffie-Hellman) with ML-KEM-768 (post-quantum). The final shared secret remains secure even if one of the two algorithms is later broken.

Authentication uses ML-DSA-65 (NIST FIPS 204) server certificates. The server signs its ML-KEM public key with its long-term ML-DSA private key. The client verifies this signature before accepting the public key, preventing man-in-the-middle substitution attacks. Tunnel traffic is encrypted by WireGuard using ChaCha20-Poly1305.

VPN Server: A self-hosted Ubuntu VPS runs WireGuard alongside our custom PQC handshake service built with the same liboqs library. Both sides must run our code to perform the ML-KEM encapsulation and decapsulation steps.

[1] Hulsing, A., Ning, K., Schwabe, P., Weber, F., & Zimmermann, P. R. (2021). Post-Quantum WireGuard. *IEEE S&P*, pp. 304-321. <https://doi.org/10.1109/SP40001.2021.00030>

3.2 Reinforcement Learning Agent

The RL agent runs locally on the user's machine as part of the VPN daemon. It is a PPO (Proximal Policy Optimisation) neural network with two hidden layers of 64 neurons each, approximately 4,932 parameters, under 20KB on disk.^{[2][3]} The trained model is shipped with the application and fine-tuned to the specific device after installation.

The agent observes seven state variables, each normalised to a 0-1 range: (1) CPU load, (2) available RAM, (3) network latency to VPN server, (4) upload traffic volume, (5) connection type (wired/WiFi/cellular), (6) time elapsed since last key exchange, and (7) a threat score produced by the anomaly detection pipeline. Based on this state vector, the agent selects one of four actions: apply ML-KEM-512, apply ML-KEM-768, apply ML-KEM-1024, or trigger an immediate key re-exchange.

The reason for dynamic rekeying is operational. Session keys sit in device RAM and can be extracted through memory attacks or side channels. Rekeying limits how much traffic is exposed if a key is ever compromised. This combination of variables is too complex for simple if-else rules to handle well, which is where the RL agent adds value.

3.3 Anomaly Detection

The threat score is computed in layers, added incrementally based on available device resources. The CPU reading used for gating is already collected by the state observer for the RL agent.

- **Layer 1 — always active (all devices):** Statistical Z-score baseline. Rolling averages of latency, packet rate, and packet sizes are maintained. Values deviating beyond three standard deviations are flagged.
- **Layer 2 — active when CPU usage below 70%:** Rule-based signature checks for known attack patterns including port scans, retransmission spikes, latency anomalies consistent with MitM insertion, bandwidth spikes indicating exfiltration, and DNS server changes. The 70% threshold follows established systems literature showing non-linear latency increases above this level [5].
- **Layer 3 — active when CPU usage below 40%:** Isolation Forest model [6] trained on normal traffic data, detecting multivariate anomalies that individual metrics miss. Only activated when the state observer confirms sufficient CPU headroom.

The entire pipeline runs once every five seconds on aggregated metrics, not per-packet. Even with all three layers active, anomaly detection cannot become a performance bottleneck on the tunnel it is protecting.

3.4 Desktop Application — Tauri

The user interface is delivered as a native desktop application built with Tauri, the same framework used by 1Password and adopted by multiple security-focused applications. Tauri was chosen over Electron deliberately: it uses the operating system's native webview. This produces an installer under 15MB compared to Electron's 150-200MB, and idle RAM usage of 20-50MB compared to Electron's 200-400MB. For a security application running alongside a Python daemon and WireGuard, this overhead difference is significant.

The application architecture has three layers. The Tauri backend is written in Rust. The frontend is a React application with Tailwind CSS and Recharts running inside the Tauri webview. The Python daemon runs as a child process managed by Tauri, exposing a FastAPI server on localhost:8000 with API key authentication on all routes.

Live statistics are delivered via WebSocket on localhost. The Python daemon pushes a state update every five seconds aligned with the RL agent loop and React re-renders on each message.

The dashboard displays: current connection status, the algorithm the agent selected, live system metrics (CPU, RAM, latency), a Recharts line graph of the last 50 agent decisions, active anomaly detection layers, time until next scheduled rekey, and a session log.

Platform delivery: Linux builds produce a .deb package and Applmage via electron-builder equivalent (Tauri's cargo tauri build). Windows builds produce an .exe NSIS installer from the same codebase by changing the build target — no code changes required between platforms.

4. Cryptographic Algorithms Used		
Algorithm	Standard	Role in the System
ML-KEM-512	NIST FIPS 203	Low-load sessions — lightweight PQ protection
ML-KEM-768	NIST FIPS 203	Default — balanced security and performance
ML-KEM-1024	NIST FIPS 203	High-sensitivity sessions, banking, elevated threat
ML-DSA-65	NIST FIPS 204	Server authentication — prevents MitM on key exchange
X25519	RFC 7748	Classical DH half of hybrid key exchange
ChaCha20-Poly1305	RFC 8439	WireGuard tunnel traffic encryption
PPO	Schulman et al. 2017 arXiv:1707.06347	RL algorithm — policy gradient agent for algorithm selection

5. How the System Works — Full Flow

The user opens the application from their Applications menu. Tauri launches a desktop window and simultaneously spawns the Python daemon as a child process. The daemon starts the FastAPI

server, loads the trained RL model into memory, and signals ready. The React dashboard connects via WebSocket and shows a disconnected state.

When the user clicks Connect, the state observer fires immediately. It reads CPU load, available RAM, network latency, and traffic metrics using psutil, and runs the anomaly detection pipeline to compute a threat score.

The PQC handler performs a hybrid X25519 + ML-KEM handshake with the VPN server after first verifying the server's ML-DSA-65 certificate. The hybrid shared secret is injected into WireGuard as the PSK using wg set. The tunnel comes up. The Tauri system tray icon turns green.

From this point the agent loop runs every five seconds. If system state changes — the user opens a banking application, moves to a public WiFi network, or a latency spike triggers an anomaly score — the agent re-evaluates and may switch algorithms or trigger a rekey. The dashboard updates in real time via WebSocket. If the agent escalates to ML-KEM-1024, a native OS notification appears immediately.

When the user disconnects or quits, the tunnel closes, all key material is securely overwritten in memory, the agent saves its experience buffer for the next session, and Tauri terminates the Python daemon cleanly.

6. Technology Stack

Desktop Application

- **Tauri** — native desktop application framework. Uses OS webview (WebKitGTK on Linux, WebView2 on Windows). Produces installers under 15MB. Idle RAM under 50MB. Chosen over Electron for lower resource overhead and stronger security isolation.
- **Rust** — Tauri backend (~50 lines). Spawns Python daemon, manages system tray, window lifecycle, and OS notifications.
- **React + TypeScript** — frontend UI running inside Tauri webview.
- **Tailwind CSS** — styling.
- **Recharts** — live charts for RL decision history and system metrics.
- **WebSocket (native browser API)** — receives live state pushes from Python daemon.

Python Daemon — RL Agent and PQC

- **Python 3.11** — primary language for daemon, RL agent, and API.
- **liboqs-python** — Open Quantum Safe library. ML-KEM-512/768/1024, ML-DSA-65.
- **cryptography (PyCA)** — X25519 for classical half of hybrid key exchange.
- **Stable-Baselines3** — PPO reinforcement learning. Default MLP policy (64x64 hidden layers, ~4,932 parameters).
- **FastAPI + uvicorn** — REST API and WebSocket server on localhost:8000. API key authenticated on all routes.
- **psutil** — system metrics (CPU, RAM, network byte counters, battery).
- **NumPy** — Z-score statistical baseline (Layer 1 anomaly detection).
- **scikit-learn** — Isolation Forest (Layer 3, activated when CPU below 40%).
- **PyTorch** — neural network backend for PPO.

VPN Tunnel

- **WireGuard** — VPN tunnel protocol. PSK injection used for PQC integration. wg-quick for tunnel management on Linux.
- **wgconfig** — Python library for reading and writing WireGuard configuration files programmatically.
- **Self-hosted Ubuntu VPS** — DigitalOcean or AWS Lightsail. Runs WireGuard and our custom PQC handshake service using the same liboqs library as the client.

Infrastructure and Tooling

- **Docker** — containerised VPN server deployment on cloud VPS.
- **GitHub** — version control. One branch per team member. Pull requests required to merge into main.
- **pytest** — unit and integration testing. All phase tests must pass before proceeding to the next phase.
- **cargo tauri build** — produces .deb and Applmage for Linux, .exe for Windows from the same codebase.

Architecture:

USER DOWNLOADS: rl-pqc-vpn.deb (Linux, ~8MB)
rl-pqc-vpn.exe (Windows, ~12MB)

ON LAUNCH:



7. What Makes This Novel

Post-quantum VPNs exist. Adaptive security systems exist. Reinforcement learning for network optimisation exists. What does not exist is a per-device RL agent learns usage patterns over time. It dynamically selects between NIST-standardised PQC algorithms based on real-time system state and detected threats.

The closest published works are a 2026 paper on context-aware PQC selection for vehicular networks [7], a 2025 paper on Q-learning driven adaptive encryption for wireless sensor networks [8], and Microsoft Research's PQC-OpenVPN fork which applies post-quantum cryptography statically [9]. None of these VPN clients with per-device reinforcement learning. That is the gap this project fills.

The academic contribution is a formally defined Markov Decision Process (MDP) for VPN security policy optimisation, where the state space captures device and network conditions, the action space covers NIST-approved algorithm choices and rekey decisions, and the reward function balances cryptographic security strength against computational overhead. The trained agent is evaluated against three baselines: classical WireGuard, static ML-KEM-768, and a rule-based policy.

8. Expected Outcomes

- A native desktop VPN application (Tauri) installable on Linux as a .deb package, with Windows .exe as a second-phase deliverable.
- A working post-quantum key exchange using hybrid X25519 + ML-KEM on real WireGuard tunnels via PSK injection, authenticated with ML-DSA-65.
- A trained PPO RL agent that demonstrably outperforms static algorithm selection and a rule-based policy baseline across at least three evaluation scenarios.
- Anomaly detection that correctly identifies at least four threat patterns in network traffic across three device capability profiles.
- Live dashboard inside the desktop application showing RL decisions, system metrics, active anomaly layers, and key rotation events — updated via WebSocket.
- A comparative evaluation measuring connection latency overhead, CPU usage, and security level against three defined baselines.
- A full project report covering system design, MDP formulation, training methodology, Tauri application architecture, and evaluation results.

References

- [1] Hulsing, A., Ning, K., Schwabe, P., Weber, F., & Zimmermann, P. R. (2021). Post-Quantum WireGuard. *IEEE Symposium on Security and Privacy (SP)*, pp. 304-321. <https://doi.org/10.1109/SP40001.2021.00030>
- [2] Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., & Dormann, N. (2021). Stable-Baselines3: Reliable Reinforcement Learning Implementations. *Journal of Machine Learning Research*, 22(268), 1-8. <https://jmlr.org/papers/v22/20-1364.html>
- [3] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal Policy Optimization Algorithms. *arXiv:1707.06347*. <https://arxiv.org/abs/1707.06347>
- [4] Donenfeld, J. A. (2017). WireGuard: Next Generation Kernel Network Tunnel. *NDSS Symposium*. <https://www.ndss-symposium.org/ndss2017/>
- [5] Urgaonkar, B., Shenoy, P., & Roscoe, T. (2002). Resource Overbooking and Application Profiling in Shared Internet Hosting Platforms. *USENIX OSDI*.
- [6] Liu, F. T., Ting, K. M., & Zhou, Z. H. (2008). Isolation Forest. *IEEE ICDM*, pp. 413-422. <https://doi.org/10.1109/ICDM.2008.17>

- [7] Alnahawi, N., et al. (2026). Context-Aware Adaptive Post-Quantum Cryptography for 6G V2X Communications. *arXiv:2602.01342*. <https://arxiv.org/abs/2602.01342>
- [8] Youssef, A., et al. (2025). Reinforcement Learning Q-Learning Adaptive Encryption in Wireless Sensor Networks. *MDPI Sensors*, 25(7), 2056. <https://doi.org/10.3390/s25072056>
- [9] Microsoft Research. Post-Quantum Cryptography VPN. <https://www.microsoft.com/en-us/research/project/post-quantum-crypto-vpn/>
- [10] NIST. (2024). FIPS 203: ML-KEM Standard. <https://csrc.nist.gov/pubs/fips/203/final>
- [11] NIST. (2024). FIPS 204: ML-DSA Standard. <https://csrc.nist.gov/pubs/fips/204/final>
- [12] NIST. (2007). SP 800-38D: GCM and GMAC. <https://csrc.nist.gov/publications/detail/sp/800-38d/final>